LAB 10: SOFTWARE METRICS & COMPLEXITY ANALYSIS

Project: Digital Health Management System (DHMS) | Software Engineering Practical

Team - 4

1. Aim

To identify and apply software metrics to the Digital Health Management System, analyse the complexity of its major modules, and define measurable indicators for evaluating software quality, testing, and maintainability.

2. Objectives

- Understand software size, functional, complexity, and quality metrics.
- Apply Cyclomatic Complexity to a representative appointment-booking operation.
- Classify DHMS modules by indicative functional complexity.
- Define metrics that can be collected during implementation and testing.
- Distinguish calculated results from values that require source code or execution data.

3. System Overview and Analysis Scope

DHMS is a role-based healthcare information and workflow system. Its scope includes user accounts and permissions, patient profiles and records, appointments, clinical encounters and care plans, prescriptions, laboratory reports, notifications and follow-up tasks, and administrative audit/reporting. The analysis below focuses on these modules as defined in the project SRS.

4. Types of Metrics Used

Metric category	What it measures	Use in DHMS
Size metrics	Amount of software or functionality	LOC/KLOC after implementation; function-point categories at requirements stage
Complexity metrics	Control-flow and logical difficulty	Cyclomatic complexity of booking, consultation, prescription, and access-control logic
Quality metrics	Reliability, correctness, maintainability, usability, and performance	Test results, defect density, response times, and requirement coverage
Process metrics	How effectively development and testing proceed	Test pass rate, defect resolution time, and review completion

Measurement note: This is a design-stage academic analysis. The project SRS describes expected features and performance targets, but no source-code repository, measured LOC, executed test results, or production telemetry were supplied for this lab. Therefore, code-dependent metrics are explained and demonstrated with a clearly labelled example, not presented as measured project results.

5. Software Metrics and Their Application

5.1 Lines of Code (LOC)

LOC counts the physical or logical lines in the implemented source code, depending on the counting convention. KLOC is calculated as LOC / 1,000. LOC can help compare implementation size within the same language and counting rules, but it does not directly measure correctness or quality.

DHMS status: Not yet measurable from the SRS. The final value depends on the selected programming language, framework, generated code, and agreed counting tool.

5.2 Function Point (FP) Analysis

Function Point Analysis estimates the functionality visible to users, independent of programming language. The system is examined using five function types:

Function type	Meaning	Examples within DHMS
External Inputs (EI)	Data or control entering the system	Registration/login submissions, profile updates, appointment requests, clinical notes, prescriptions, lab uploads
External Outputs (EO)	Processed information leaving the system	Appointment confirmations, reminders, prescription views, lab report delivery, operational reports
External Inquiries (EQ)	Request/response retrieval with limited processing	Appointment availability search, patient history lookup, appointment/report status queries
Internal Logical Files (ILF)	Logical data maintained by the application	Users/roles, patient records, appointments, encounters, prescriptions, medication catalogue, lab orders/reports, notifications, audit events
External Interface Files (EIF)	Data maintained by an external system but referenced by DHMS	External identity provider, laboratory/FHIR system, notification provider, optional billing/pharmacy service

FP calculation approach: Count each function, rate its complexity using the required data-element and file-reference rules, then multiply by the applicable standard weight. Sum the weighted counts to obtain Unadjusted Function Points (UFP). A defensible numeric FP total cannot be calculated from the high-level SRS alone because detailed transaction layouts, data-element counts, and referenced logical files have not been fully specified.

5.3 Cyclomatic Complexity (CC)

Cyclomatic Complexity estimates the number of linearly independent paths through a program control-flow graph. It helps identify routines that may need more tests or decomposition.

Formula: M = E - N + 2P, where E is the number of control-flow edges, N is the number of nodes, and P is the number of connected components. For one connected routine, M is also equal to the number of decision points + 1.

5.4 Halstead Metrics

Halstead metrics use counts of operators and operands in source code. Let n1 = distinct operators, n2 = distinct operands, N1 = total operator occurrences, and N2 = total operand occurrences. Vocabulary n = n1 + n2; program length N = N1 + N2; Volume V = N log2(n); Difficulty D = (n1 / 2) x (N2 / n2); Effort E = D x V. These values require actual source code and are not calculated at this design stage.

6. Worked Cyclomatic Complexity Analysis

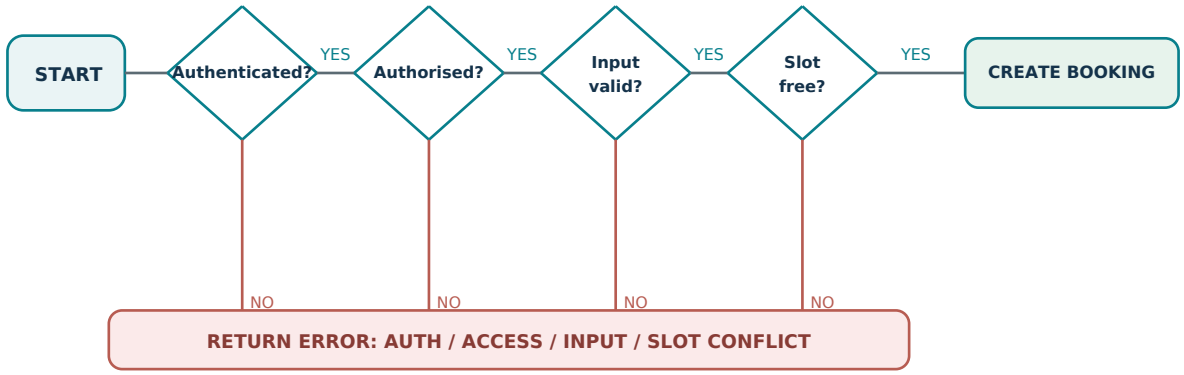
Representative operation: Book Appointment. The flow checks identity, authorization, request validity, and slot availability before creating an appointment. It reflects the booking rules in the DHMS SRS (appointment availability, unique booking, prevention of double booking, conflict response, and audit/notification behavior).

6.1 Pseudocode

```
FUNCTION bookAppointment(request, user):
IF user is not authenticated: RETURN AUTH_FAILED
IF user is not authorized: RETURN ACCESS_DENIED
IF request is invalid: RETURN INVALID_REQUEST
IF requested slot is not available: RETURN SLOT_CONFLICT
appointment = create appointment(request)
queue confirmation notification(appointment)
write audit event(appointment, user)
RETURN CONFIRMED
```

6.2 Control-Flow Diagram

The four diamonds are decision points. Each has a successful path to the next check and an alternate path to an error result.



Each decision has a YES path to the next check and a NO path to an error result.

6.3 Calculation

Item	Value	Explanation
Decision points	4	Authenticated? Authorised? Input valid? Slot free?
Connected components (P)	1	One connected control-flow graph
Cyclomatic complexity	M = 4 + 1 = 5	Decision points + 1 for a single connected routine
Interpretation	Low / manageable	Using the common guideline that values 1-10 are relatively simple; thresholds are guidelines, not universal rules.

6.4 Independent Basis Paths / Test Cases

Test path	Expected result
1. Authentication check fails	AUTH_FAILED; no appointment is created.
2. Authenticated, but permission check fails	ACCESS_DENIED; attempt is handled according to access policy.
3. Identity and permission pass, input is invalid	INVALID_REQUEST; validation feedback is returned.
4. Input is valid, but slot is unavailable	SLOT_CONFLICT; user is asked to choose another slot.
5. All checks pass	Appointment is created; notification is queued and audit event is recorded.

Result: The representative booking routine has Cyclomatic Complexity 5 and requires at least five basis-path tests to exercise the independent control-flow paths. Additional tests are still needed for boundaries, concurrency, external-service failures, and security cases.

7. Module-wise Complexity Analysis

The following ratings are qualitative design estimates based on the number of workflows, validation rules, state changes, integrations, and security constraints described in the SRS. They are not measured Cyclomatic Complexity values from implementation code.

DHMS module	Indicative complexity	Reason	Design / testing focus
User account & access management	Medium-High	Authentication, role assignments, session handling, and authorization checks	Test every role/action combination; deny-by-default behavior
Patient profile & health records	Medium-High	Multiple linked data types, validation, privacy, and record history	Field validation, ownership checks, data integrity, and auditability
Appointment management	High	Availability, rescheduling, cancellation, double-booking prevention, and status transitions	Overlap, concurrency, boundary-time, cancellation, and no-show tests
Clinical documentation & care plans	High	Draft/finalized states, author metadata, amendments, and clinical workflow	Role checks, finalization rules, amendment history, and data completeness
Prescription & medication management	High	Medicine instructions, required-field validation, allergies, and prescription status	Dosage-field validation, allergy visibility, status and amendment tests
Laboratory & diagnostic reports	Medium-High	Order/report relationships, file uploads, provenance, review, and access control	Upload validation, file integrity, pending/finalized access rules
Notifications & follow-up	Medium	Channel preferences, retries, delivery status, reminders, and due tasks	Provider outage, retry/idempotency, preference, and failure handling
Administration, audit & reporting	Medium-High	Privileged configuration, audit review, exports, and operational dashboards	Admin permission tests, audit completeness, export scope, and configuration validation

8. Software Quality and Testing Metrics

Metric	Formula / measurement	How DHMS should use it
Requirement test coverage	(Requirements with at least one test / total testable requirements) x 100	Track coverage of the 56 functional requirements in the SRS; report uncovered requirements.
Test pass rate	(Passed tests / executed tests) x 100	Report by module and release; do not hide blocked or skipped tests in the denominator.
Defect density	Confirmed defects / KLOC	Calculate after code and defect records exist; compare only using consistent tools and rules.
Mean time to resolve	Total time to resolve defects / number of resolved defects	Monitor how quickly high-priority defects are fixed and verified.
Response time	Measure request latency under defined load	SRS targets: standard authenticated page/API response within 2 seconds under nominal load; 95% of interactive requests within 4 seconds, excluding third-party delays.
Availability	Uptime / (uptime + downtime) x 100	Measure over an agreed monitoring window and define planned maintenance treatment.

9. Interpretation and Recommendations

The design-stage analysis indicates that appointment scheduling, clinical documentation, and prescription management deserve the most focused complexity and test-design attention. These modules contain multiple state transitions, business validations, and safety/privacy-sensitive operations. Authentication and access control are cross-cutting concerns and must be tested across all modules.

9.1 Recommendations for Reducing Complexity

• Keep each service focused on one responsibility; separate appointment availability, booking, notification, and audit operations.
• Use explicit state machines for appointments, encounters, prescriptions, and report review status.
• Centralize authorization checks and apply them on the server for every protected operation.
• Extract complex validation into small, named functions and test each function independently.
• Avoid deeply nested conditional logic; use clear guard clauses and well-defined error outcomes.
• Keep external integrations behind adapters so failures in email, SMS, laboratory, or billing services do not corrupt core records.
• Preserve audit history for sensitive access and material clinical changes.

9.2 Metrics to Collect During Implementation

When	Metrics to collect	Evidence / tool
During coding	LOC/KLOC, Cyclomatic Complexity per function, Halstead counts where useful	Repository, static-analysis tools, complexity reports
During testing	Requirement coverage, test pass rate, defects by severity, regression failures	Test management records and CI reports
During performance testing	Average and percentile response time, throughput, error rate	Load-test results under documented test conditions
During pilot / operation	Availability, recovery time, follow-up completion, appointment conflicts, access-control events	Monitoring dashboards and approved operational reports

10. Result and Conclusion

Software metrics provide a systematic way to assess the size, complexity, quality, and maintainability of DHMS. Function Point Analysis can describe the functional scope at the requirements stage, while LOC and Halstead metrics require implemented source code. Cyclomatic Complexity was calculated for a representative appointment-booking routine: four decision points produce a value of 5, with five independent basis paths. The module-wise review identifies appointment, clinical documentation, and prescription workflows as areas needing careful decomposition and thorough testing.

The analysis is preliminary and should be updated when implementation code, detailed function-point counts, test execution results, and operational measurements become available. No numeric code-size, defect-density, test-pass, or production-availability result is claimed in this report.

11. References

1. Digital Health Management System, Software Requirements Specification (SRS), Version 1.0, especially Sections 2, 4, and 5 and Appendix B.
2. DHMS project case-study presentation and previous laboratory models: class, activity, data-flow, sequence, state-transition, and component diagrams.
3. McCabe, T. J., “A Complexity Measure,” IEEE Transactions on Software Engineering, 1976 (Cyclomatic Complexity concept).

Key takeaway: Measure what is implemented, test the paths that matter, and keep high-risk healthcare workflows small, traceable, and auditable.